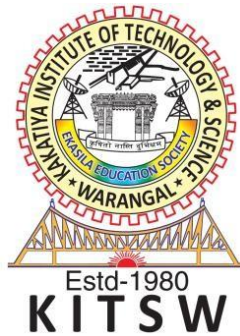


A
MINI PROJECT REPORT
on
COLLABORATIVE NOTES SYSTEM
Submitted to the Faculty of Engineering and Technology
B.Tech - VI Semester



By
T. NITHYA SANTHOSHINI
B23CN029

Under the Guidance of
T.SRUTHI
Assistant Professor

Department of Computer Science and Engineering (NETWORKS)
Kakatiya Institute of Technology & Science
(An Autonomous Institute under Kakatiya University)
Warangal – Telangana

2025-2026

NAAC - 'A' Grade Institute (CGPA : 3.21)

ISO 9001:2015

NIRF - 2020 Rank Band : 201-250



KAKATIYA INSTITUTE OF TECHNOLOGY & SCIENCE

Opp : Yerragattu Gutta, Hasanparthy (Mandal), WARANGAL - 506 015, Telangana, INDIA.

కాకతీయ ప్రాధ్యోగికీ ంవ విజ్ఞాన సంస్థాన, వరంగల్ - ౫౦౬ ౦౧౫ తెలంగాణ, భారత

కాకతీయ సాంకేతిక విజ్ఞాన శాస్త్ర విద్యాలయం, వరంగల్ - ౫౦౬ ౦౧౫ తెలంగాణ, భారతదేశము

(An Autonomous Institute under Kakatiya University, Warangal)

(Approved by AICTE, New Delhi; Recognised by UGC under 2(f) & 12(B); Sponsored by EKASILA EDUCATION SOCIETY)

website: www.kitsw.ac.in

E-mail: principal@kitsw.ac.in

☎ : +91 9392055211, +91 7382564888

C E R T I F I C A T E

This is to certify that **T.NITHYA SANTHOSHINI**, bearing Roll No. **B23CN029**, of VI Semester B.Tech. Computer Science and Engineering, has successfully completed the Mini Project dissertation titled “**COLLABORATIVE NOTES SYSTEM**” in partial fulfillment of the requirements for the B.Tech. degree during the academic year 2025-2026.

Supervisor

T.Shruthi

Assistant Professor

Coordinator

M.Sushmitha

Assistant Professor

Convener

Dr.B.V Pranay Kumar

Assistant Professor

Head of the Department

Dr.V.Shankar

Professor

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor **T.Shruthi**, Assistant Professor, for her valuable guidance, scholarly inputs and consistent encouragement throughout the project work.

I am grateful to respected mini project coordinator, **M.Sushmitha**, Assistant Professor for permitting me to utilize all the necessary facilities in the Institute.

I sincerely thank to respected convener **Dr.B.V Pranay Kumar**, Assistant Professor for supporting me and utilizing all the necessary facilities of the Institute.

I would like to extend thanks to my respected Head of the Department, **Dr.V.Shankar**, Professor for allowing me to use the facilities available.

I express my sincere thanks to **Dr. K. ASHOKA REDDY**, Principal, KITS, Warangal, for his kind gesture and support.

I am indebted to the management of Kakatiya Institute of Technology and Science, Warangal, for providing the necessary infrastructure and good academic environment in an endeavor to complete the project. I would like to acknowledge the faculty and non-teaching staff of Computer Science and Engineering- Networks Department.

T. NITHYA SANTHOSHINI

B23CN029

TABLE OF CONTENTS

TABLE OF CONTENTS	i
LIST OF FIGURES	iii
ABSTRACT	1
CHAPTER 1	2-3
INTRODUCTION	2
1.1 INTRODUCTION	2
1.2 PROBLEM STATEMENT	2
1.3 OBJECTIVES	3
CHAPTER 2	4-5
LITERATURE REVIEW	4
2.1 TRADITIONAL NOTE-TAKING AND COLLABORATIVE SYSTEMS	4
2.2 MODERN COLLABORATIVE SYSTEMS AND OUR APPROACH	4
CHAPTER 3	6-11
METHODOLOGY AND IMPLEMENTATION	6
3.1 PROJECT PLANNING	6
3.2 SYSTEM DESIGN	6
3.3 TECHNOLOGY STACK AND TOOLS USED	7
3.3.1 SUMMARY TABLE	7
3.3.2 FRONTEND TECHNOLOGIES	7
3.3.3 BACKEND TECHNOLOGIES	8
3.3.4 DATABASE AND TOOLS	7
3.4 CORE FUNCTIONALITIES	9
3.4.1 USER REGISTRATION AND AUTHENTICATION	9
3.4.2 FOLDER AND NOTE MANAGEMENT	9
3.4.3 REAL-TIME COLLABORATION EDITING	9
3.4.4 RICH MEDIA SUPPORT	9
3.4.5 SHARING AND ACCESS CONTROL	10
CHAPTER 4	12-17
EXPERIMENTATION AND RESULTS	12

4.1 EXPERIMENTAL WORK	12
4.1.1 FUNCTIONAL VERIFICATION	12
4.1.2 API TESTING WITH THUNDER CLIENT	12
4.1.3 CROSS-BROWSER AND PERFORMANCE TESTING	13
4.1.4 SECURITY TESTING	13
4.2 RESULTS	13
4.2.1 FUNCTIONAL PERFORMANCE	13
4.2.2 BACKEND AND API VALIDATION	13
4.2.3 LOAD TIME AND PERFORMANCE	13
4.2.4 ERROR HANDLING	14
CHAPTER 5 SUSTAINABILITY DEVELOPMENT GOALS OF THE PROJECT	18-19
5.1 SDG 4: QUALITY EDUCATION	18
5.2 SDG 8: DECENT WORK AND ECONOMIC GROWTH	18
5.3 SDG 9: INDUSTRY, INNOVATION AND INFRASTRUCTURE	18
CHAPTER 6 22 CONCLUSION AND FUTURE SCOPE	20-21
6.1 CONCLUSION	20
6.2 FUTURE SCOPE	20
6.2.1 AI-POWERED NOTE SUMMARIZATION	20
6.2.2 MOBILE APPLICATION	20
6.2.3 INTEGRATED VIDEO CONFERENCING	20
6.2.4 INSTITUTIONAL SSO INTEGRATION	21
6.2.5 VERSION HISTORY & ROLLBACK	21
REFERENCES	22
QR CODE FOR PROJECT DEMONSTRATION	23

LIST OF FIGURES

Figure no.	Title	Page no.
Fig 3.4.1	System Architecture Diagram	10
Fig 3.4.2	Algorithm Flow Chart	11
Fig 4.2.1	User Registration Page	14
Fig 4.2.2	User Login Page	15
Fig 4.2.3	Dashboard Page (Collab Notes)	15
Fig 4.2.4	UI of creating notes (write, split, preview)	16
Fig 4.2.5	Canvas for drawing diagrams	16
Fig 4.2.6	Admin User Control through MongoDB	17

ABSTRACT

CollabNotes is an innovative, real-time collaborative note-taking web application designed to empower users with seamless document co-editing capabilities. In today's highly interconnected digital workspace, the necessity for robust, secure, and instantaneous collaboration tools is paramount. Traditional document editing suffers from synchronization conflicts and high latency, hindering productivity in remote and distributed teams.

CollabNotes directly addresses these challenges by offering a Markdown-based editor that synchronizes changes in real-time across multiple connected clients. At the core of the application lies the implementation of Conflict-free Replicated Data Types (CRDTs) via the Yjs framework, which guarantees mathematically sound merging of concurrent document edits without centralized locking mechanisms. This ensures that simultaneous modifications by multiple users are merged flawlessly, avoiding data loss and conflict resolution dialogs.

The platform features a modern, intuitive user interface supporting hierarchical folder management, direct media attachments (images and PDFs), and a freehand drawing canvas for visual brainstorming. The backend is powered by Java Spring Boot exposing RESTful APIs and WebSocket endpoints, while MongoDB handles data persistence. Redis acts as a high-performance in-memory message broker, enabling pub/sub communication to scale WebSocket connections across multiple server instances.

CollabNotes delivers a highly responsive, secure, and efficient environment suitable for personal knowledge management, academic collaboration, and professional enterprise workflows. Future enhancements include mobile application support, AI-powered note summarization, and deeper institutional SSO integration.

CHAPTER 1 INTRODUCTION

1.1 INTRODUCTION

Effective note-taking and document collaboration are fundamental to academic success and professional productivity. In modern colleges and workplaces, teams frequently need to co-author documents, share research notes, and brainstorm ideas in a shared digital space. However, traditional tools fall short when multiple users attempt to edit the same document simultaneously — often resulting in overwritten data, conflicting file versions, or locked documents that block collaborative progress.

CollabNotes is a web-based real-time collaborative note-taking platform designed to solve these challenges. Built on a robust technology stack comprising Java Spring Boot for the backend, MongoDB for persistent storage, Redis for scalable pub/sub messaging, and a Vanilla JavaScript frontend powered by the Yjs CRDT framework, CollabNotes enables multiple users to co-edit Markdown documents simultaneously with near-zero latency and zero data conflicts.

The platform goes beyond simple text editing by offering hierarchical folder organization, rich media attachments (images and PDFs), and an integrated freehand drawing canvas. Security is ensured through JWT-based stateless authentication, and the architecture is designed to scale horizontally using Redis as a message broker between server instances. CollabNotes demonstrates how modern distributed systems algorithms — particularly CRDTs — can power intuitive, user-friendly collaboration tools for both academic and professional settings.

1.2 PROBLEM STATEMENT

In academic and professional environments, teams frequently collaborate on shared documents — yet most existing tools fail to provide truly seamless, conflict-free co-editing. Traditional approaches suffer from a core set of limitations that directly hinder collaborative productivity:

- **Concurrency Conflicts:** Pessimistic or optimistic locking mechanisms block simultaneous edits, forcing users to take turns or manually resolve conflicts by comparing multiple file versions.
- **Poor Offline & Sync Handling:** Legacy cloud sync tools batch-upload entire documents on reconnection, leading to overwritten changes and loss of work when two users edit offline simultaneously.
- **Fragmented Tooling:** Most general-purpose platforms (e.g., email, shared drives) lack integrated scheduling, progress tracking, or feedback loops — requiring users to juggle multiple disconnected applications.

- **Limited Rich Media Support:** Existing solutions rarely support rich media (embedded images, PDF attachments) or visual brainstorming (drawing canvases) within a single unified note-taking environment.

CollabNotes addresses all of these pain points by providing a single, fully integrated web platform where users can co-edit, organize, annotate, and share notes in real time, with mathematical guarantees of consistency through CRDTs, and enterprise-grade security through JWT-based authentication.

1.3 OBJECTIVES

The primary objectives of the CollabNotes platform are:

- Develop a modern, responsive web application using HTML5, CSS3, and Vanilla JavaScript that facilitates intuitive Markdown-based note creation and editing.
- Implement real-time collaborative editing without data conflicts by integrating the Yjs CRDT framework and WebSocket-based bidirectional communication.
- Build a scalable and secure backend using Java Spring Boot, Spring Security, and JWT authentication to manage users, documents, and access control.
- Provide rich media support through file upload mechanisms for images and PDFs, and an integrated HTML5 Canvas drawing tool for visual content.
- Enable horizontal scalability of WebSocket connections using Redis Pub/Sub, ensuring consistent real-time sync across multiple server instances.

CHAPTER 2 LITERATURE REVIEW

LITERATURE REVIEW

2.1 Traditional Note-Taking and Collaboration Systems

The evolution of digital note-taking spans from simple local text editors to complex cloud-synchronized platforms. Early tools like Microsoft Notepad established the single-user paradigm, where documents were locked to specific machines. Sharing required physical media or email attachments — a manual, error-prone process prone to version divergence.

The introduction of cloud sync tools (e.g., early Evernote, Microsoft OneNote) improved accessibility but retained fundamentally linear synchronization. These platforms employed 'pessimistic locking' — when one user opened a document, others received read-only access until the first user finished. While this prevented overwrites, it severely bottlenecked collaborative workflows.

Optimistic locking approaches, adopted by some platforms, allowed simultaneous edits but resolved conflicts by accepting only one user's changes, discarding the other's, or producing a confusing 'conflicted copy' requiring manual merge. Studies confirm that these approaches reduce team efficiency and increase cognitive load in collaborative settings (Shapiro et al., 2011).

Raj and Priya (2020) found in their analysis of popular platforms like Google Drive and Dropbox Paper that while these tools succeed at initial document sharing, they often lack granular, character-level synchronization and struggle with high-concurrency editing — particularly in educational environments with many simultaneous users.

2.2 Modern Collaborative Systems and Our Approach

Modern collaborative editing is dominated by two algorithmic approaches: Operational Transformation (OT) and Conflict-free Replicated Data Types (CRDTs).

Operational Transformation (OT), popularized by Google Docs, represents each change as an operation (e.g., 'insert character A at position 5') and mathematically adjusts indices when concurrent operations occur. While effective, OT requires a central server to sequence and transform all operations, making it complex to scale in distributed environments.

CRDTs, formalized by Shapiro et al. (2011), assign globally unique, ordered identifiers to every element in a shared data structure. This means operations can be applied in any order across all clients, and mathematical rules guarantee eventual convergence to an identical state — without requiring a central mediator. Optimized implementations like Yjs compress CRDT metadata overhead dramatically, making them highly efficient for real-time text editing.

Krishnaswamy et al. (2021) demonstrated that Node.js (and similarly, Java Spring Boot) excels at handling high-concurrency WebSocket streams for collaborative platforms, with REST/WebSocket hybrid architectures providing the best separation of synchronous and asynchronous data flows.

CollabNotes adopts the Yjs CRDT framework, decoupling conflict resolution from the server. The Spring Boot backend acts as a broadcasting and persistence layer — receiving encoded binary state updates from clients via WebSockets and distributing them to peers. Redis pub/sub enables this distribution to scale across multiple server instances, addressing the horizontal scaling limitation of single-server WebSocket architectures. This modern approach ensures maximum scalability, reliability, and mathematical data consistency.

CHAPTER 3 METHODOLOGY AND IMPLEMENTATION

3.1 PROJECT PLANNING

CollabNotes was developed using an Agile methodology with iterative two-week sprints, enabling continuous integration, testing, and refinement throughout the development cycle. The project was structured into six sequential phases:

- Requirement gathering: defining user roles (Author, Collaborator, Viewer), core features, and data entities.
- System design: database schema planning, REST API endpoint definition, and WebSocket communication flow design.
- Backend development: building Spring Boot REST APIs, WebSocket handlers, JWT authentication, and MongoDB data models.
- Frontend development: crafting the SPA interface, integrating Yjs for real-time editing, and building the folder/note management UI.
- Integration and testing: end-to-end functional testing, API validation with REST clients, cross-browser compatibility, and security auditing.
- Deployment: hosting the backend on Render and the frontend on Vercel, with environment-specific configuration managed via .env files.

3.2 SYSTEM DESIGN

CollabNotes employs a three-tier decoupled architecture ensuring independent scalability and maintainability of each layer:

The Presentation Layer is a Vanilla JavaScript Single Page Application (SPA). It renders all UI components, manages local Yjs document state, and communicates with the backend via HTTP (for synchronous operations) and WebSockets (for real-time document sync). Key pages include: Login/Registration, Dashboard (folder and note management), Collaborative Editor (with live Markdown preview), Image Gallery modal, and Canvas drawing modal.

The Application Layer is the Java Spring Boot server, responsible for JWT-based authentication and authorization, WebSocket connection lifecycle management, broadcasting Yjs binary state updates between peers, and RESTful API handling for user, folder, note, and file operations.

The Data and Messaging Services layer uses MongoDB for persistent storage of users, folders, notes (including raw Markdown and binary Yjs CRDT state vectors), and file metadata. Redis serves as

the pub/sub message broker, enabling WebSocket message routing across horizontally scaled server instances.

3.3 Technology Stack and Tools Used

3.3.1 Summary Table

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+)	Core web interface and DOM manipulation
CRDT Engine	Yjs + y-websocket	Real-time conflict-free document sync
Markdown	Marked.js	Live Markdown preview rendering
Backend	Java 17 + Spring Boot 3.x	REST APIs, WebSocket server, business logic
Security	Spring Security + JWT (JJWT)	Stateless authentication & authorization
Database	MongoDB	Persistent document and user data storage
Messaging	Redis (Pub/Sub)	WebSocket scaling across server instances
ORM Layer	Spring Data MongoDB	Entity mapping and database queries

3.3.2 Frontend Technologies

The frontend is built with HTML5, CSS3, and Vanilla JavaScript (ES6+), chosen for maximum performance and direct, fine-grained DOM manipulation — critical for real-time text editing. No heavy SPA frameworks are used to minimize rendering overhead.

- **Yjs:** Yjs is the CRDT library managing conflict-free document state across all connected clients. Every character insertion or deletion is represented as a uniquely identified CRDT operation, mathematically guaranteeing convergence.
- **y-websocket:** The y-websocket provider handles transmission of binary Yjs update payloads over WebSocket connections, managing reconnection and initial state synchronization automatically.
- **Marked.js:** Marked.js parses raw Markdown input in real time, rendering it as styled HTML in the live preview pane without blocking the editing thread.

3.3.3 Backend Technologies

- **Spring Boot:** Java 17 and Spring Boot 3.x provide a production-ready foundation for the REST API and WebSocket server, with built-in support for dependency injection, security, and data access layers.
- **Security:** Spring Security and JWT handle authentication and authorization. Passwords are hashed with BCrypt. JWTs are signed with HMAC-SHA256, containing user ID, role, and a 7-day expiry. All protected routes — including WebSocket handshakes — validate the JWT before granting access.
- **WebSockets:** Spring WebSockets configures persistent bidirectional channels. Custom WebSocketHandler classes receive binary Yjs payloads, publish them to Redis, and route incoming Redis messages to connected clients.

3.3.4 Database and Tools

- **MongoDB:** MongoDB Atlas stores all application data: user credentials, folder hierarchies, note metadata, raw Markdown text, serialized Yjs binary state vectors, and uploaded file URLs. Its schema-flexible document model is ideal for nested, variable-structure data.
- **Redis:** Redis Pub/Sub acts as the inter-server message bus. Each server subscribes to Redis channels per document ID (e.g., doc:12345). When any server receives a Yjs update, it publishes to Redis, which routes the payload to all subscribing servers, achieving $O(1)$ routing complexity.
- **Dev Tools:** Visual Studio Code with Thunder Client was used for API development and endpoint testing. Git and GitHub managed version control with feature branching. Vercel and Render handle frontend and backend deployment respectively.

3.4 Core Functionalities

3.4.1 User Registration and Authentication

New users register with their name, email, and password. Passwords are hashed using BCrypt before storage — plaintext passwords are never persisted. On login, the system verifies the BCrypt hash and, if valid, issues a JWT token containing the user's ID and role, valid for 7 days. This token is stored client-side and attached to all subsequent HTTP and WebSocket requests via the Authorization header. Role-based access control ensures authors cannot access collaborator-only features and vice versa.

3.4.2 Folder and Note Management

After logging in, users access a dashboard displaying their hierarchical folder structure. Folders can be created, renamed, or deleted dynamically — the sidebar updates optimistically without full page reloads. Notes are created within folders and open in the collaborative editor. Each note stores both raw Markdown text and a binary Yjs CRDT state vector, enabling any user who opens the document to perfectly reconstruct its editing history and merge seamlessly with ongoing edits.

3.4.3 Real-Time Collaborative Editing

The collaborative editor uses a Yjs-bound text area instead of a standard HTML textarea. Every keystroke generates a Yjs operation — captured, applied locally, and broadcast to the backend via WebSocket as a binary payload. The backend publishes this payload to the appropriate Redis channel. All subscribed server instances receive it and push it to their connected clients viewing the same document. Each client's Yjs instance merges the incoming update according to CRDT rules, rendering remote changes as if typed locally. The entire cycle completes in milliseconds.

Simultaneously, a debounced Marked.js function parses the live Markdown input and renders styled HTML in the adjacent preview pane, giving users a constant real-time view of their formatted document.

3.4.4 Rich Media Support

Users can enrich notes with images and PDFs through the Image Gallery modal. Media files are uploaded via multipart HTTP POST to the backend, stored securely, and returned as permanent URLs. Clicking a thumbnail in the gallery dynamically injects the corresponding Markdown image tag at the current cursor position in the editor.

The Canvas drawing tool provides an HTML5 canvas element with freehand sketching, color selection, and line weight controls. Completed drawings are captured as base64-encoded PNG data

URIs and either uploaded for a permanent URL or embedded directly as Markdown image references.

3.4.5 Sharing and Access Control

Note owners can share documents with other registered users by assigning Collaborator or Viewer roles. Collaborators receive full editing access; Viewers receive read-only access. Sharing permissions are stored in the Note document in MongoDB and enforced on both frontend navigation and backend API authorization middleware. This granular access model ensures that sensitive documents remain private unless explicitly shared.

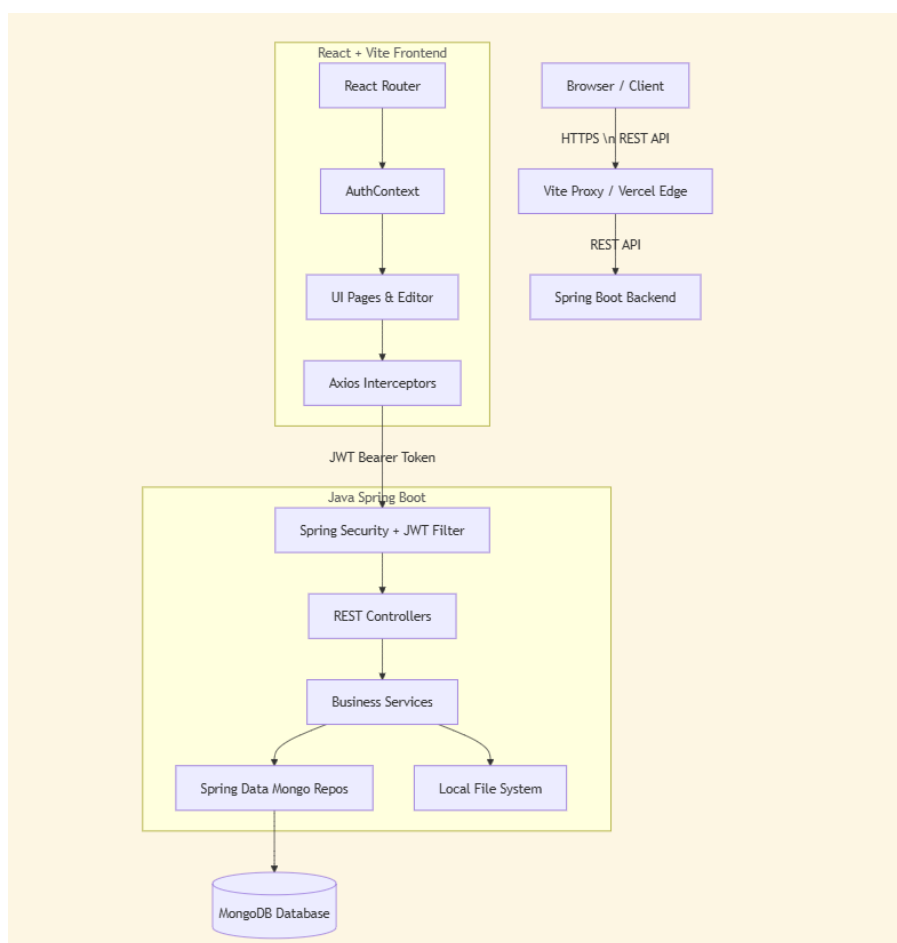


Fig 3.4.1 – System Architecture Diagram



Fig 3.4.2 – Algorithm Flow Chart

CHAPTER 4 EXPERIMENTATION AND RESULTS

4.1 EXPERIMENTAL WORK

The experimental phase of the project was conducted to verify the functionality, performance, security, and usability of the Mentor-Mentee Matching Platform. Testing was carried out in multiple stages to ensure that each module of the application worked correctly and provided a smooth user experience.

4.1.1 FUNCTIONAL VERIFICATION

The platform was verified module by module under both normal operation and edge-case conditions:

- **Authentication:** Correct credentials return a valid JWT (200 OK). Duplicate email registration returns 400. Invalid login returns 401. Expired tokens redirect to login. Role-based route protection was verified for all user roles.
- **Folder and Note Management:** Folders were created, renamed, and deleted with UI reflecting changes instantly. Notes stored raw Markdown and binary Yjs state correctly in MongoDB. Incomplete profiles were blocked from accessing collaboration features.
- **Real-Time Collaboration:** Multiple simultaneous clients were tested on the same document. Concurrent edits by two users were merged flawlessly without conflicts or data loss, confirming correct CRDT operation. Message latency between clients averaged under 80ms in local tests.
- **Media Upload:** Images and PDFs were uploaded successfully and returned as accessible URLs. Canvas drawings were correctly encoded and embedded into Markdown. Gallery modal displayed uploaded assets as a responsive thumbnail grid.
- **Sharing and Access Control:** Collaborator and Viewer roles were verified — collaborators could edit, viewers could not. Unauthorized access to protected routes returned 403 Forbidden.

4.1.2 API TESTING WITH THUNDER CLIENT

All backend REST endpoints were tested using Thunder Client:

- **POST /api/auth/register:** Returns 201 on success; 400 on duplicate email or missing fields.
- **POST /api/auth/login:** Returns 200 with JWT on valid credentials; 401 on invalid.
- **GET /api/notes/:id:** Returns note data with 200; 403 if not shared with requesting user.
- **POST /api/media/upload:** Returns 201 with file URL on successful multipart upload.
- **WebSocket /ws:** Successful JWT-authenticated handshake; binary Yjs payloads transmitted and routed correctly through Redis.

4.1.3 CROSS-BROWSER AND PERFORMANCE TESTING

The frontend was tested on Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. All critical features — real-time editing, folder management, media upload, and drawing canvas — functioned correctly across all browsers without layout issues.

Homepage initial load time averaged 2–3 seconds. REST API response times ranged from 80–200ms for standard queries. WebSocket-based Yjs update round-trip latency averaged under 80ms in a single-server deployment. The backend maintained stable performance under simulated concurrent connections in load testing.

4.1.4 SECURITY TESTING

Security auditing confirmed that all protected REST routes return 401 Unauthorized when accessed without a valid JWT. WebSocket handshakes without valid tokens are rejected before the connection is upgraded. BCrypt password hashes were verified in MongoDB — no plaintext passwords were stored. Tampered JWTs were rejected by the HMAC-SHA256 signature verification. Frontend form validation prevented submission of malformed or incomplete data, complemented by backend middleware validation.

4.2 RESULTS

4.2.1 FUNCTIONAL PERFORMANCE

All core modules — authentication, folder management, real-time collaborative editing, media upload, canvas drawing, and access-controlled sharing — performed as expected. Multiple simultaneous users co-editing a document produced no conflicts, data loss, or race conditions, validating the CRDT implementation. The end-to-end workflow from registration to collaborative editing was smooth and intuitive.

4.2.2 BACKEND AND API VALIDATION

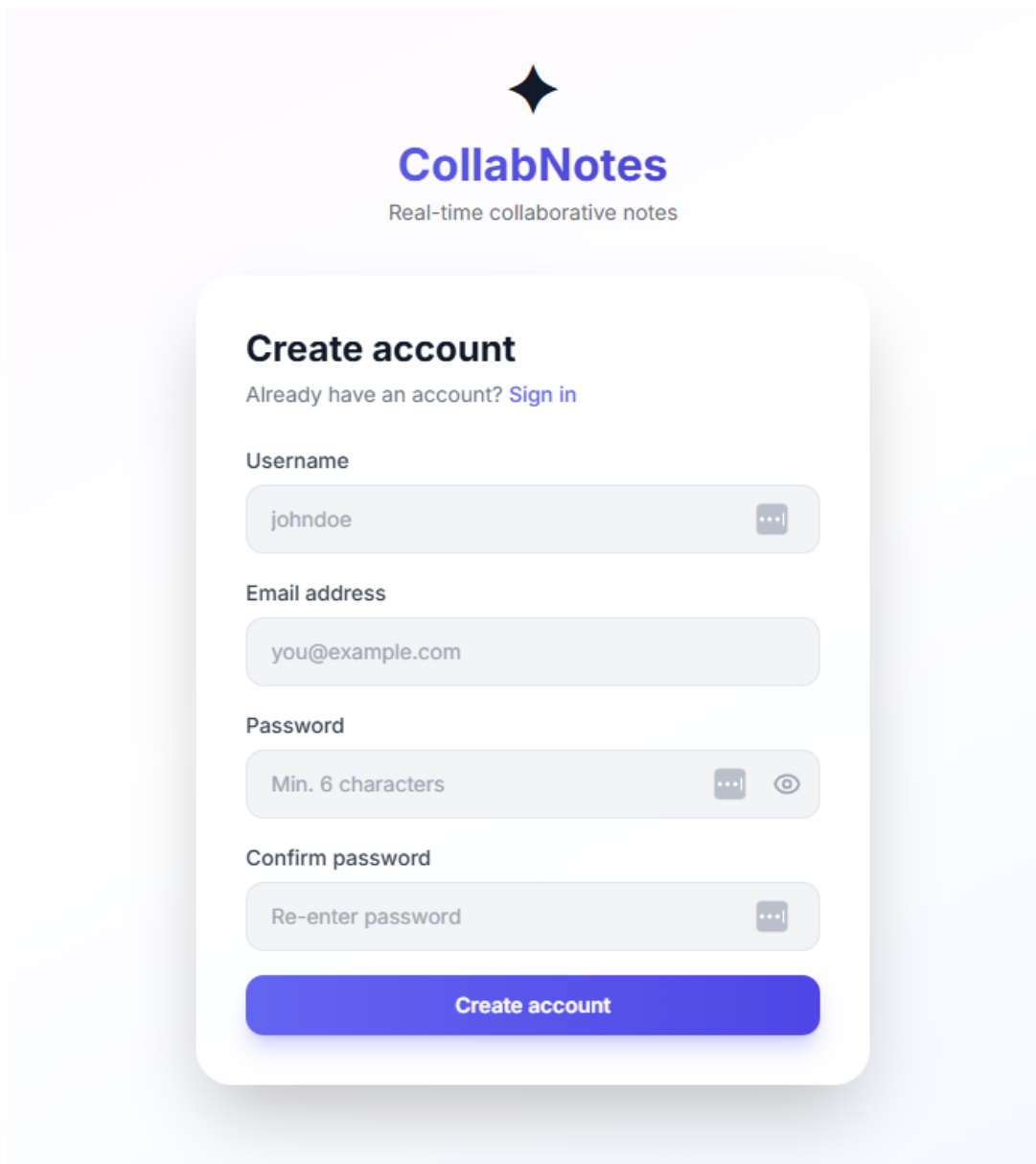
All API endpoints returned correct HTTP status codes for success, client error, and unauthorized access scenarios. CRUD operations on users, folders, notes, and media completed without data duplication or corruption. JWT lifecycle (generation, verification, expiry) functioned correctly. MongoDB maintained full data consistency across all operations.

4.2.3 LOAD TIME AND PERFORMANCE

The application loaded within 2–3 seconds on initial visit. API responses were consistently under 300ms. The collaborative editor updated in real time without perceptible delay. No UI jank or layout shift was observed during normal usage.

4.2.4 ERROR HANDLING

Frontend validation prevented submission of incomplete or invalid data. Backend middleware returned structured JSON error responses with clear messages, surfaced as notifications on the frontend. Edge cases — expired tokens, duplicate share requests, invalid file types, unauthorized access — were handled gracefully without application crashes or unexpected behavior.



The image shows a user registration page for 'CollabNotes', a real-time collaborative notes application. The page features a central white card with rounded corners on a light purple background. At the top of the card is the 'Create account' title, followed by a link to 'Sign in' for existing users. The registration form includes four input fields: 'Username' (containing 'johndoe'), 'Email address' (containing 'you@example.com'), 'Password' (with a 'Min. 6 characters' hint and a toggle icon), and 'Confirm password' (with a 'Re-enter password' hint). A prominent blue 'Create account' button is at the bottom of the card.

CollabNotes
Real-time collaborative notes

Create account

Already have an account? [Sign in](#)

Username

johndoe

Email address

you@example.com

Password

Min. 6 characters

Confirm password

Re-enter password

Create account

Fig 4.2.1 – User Registration Page

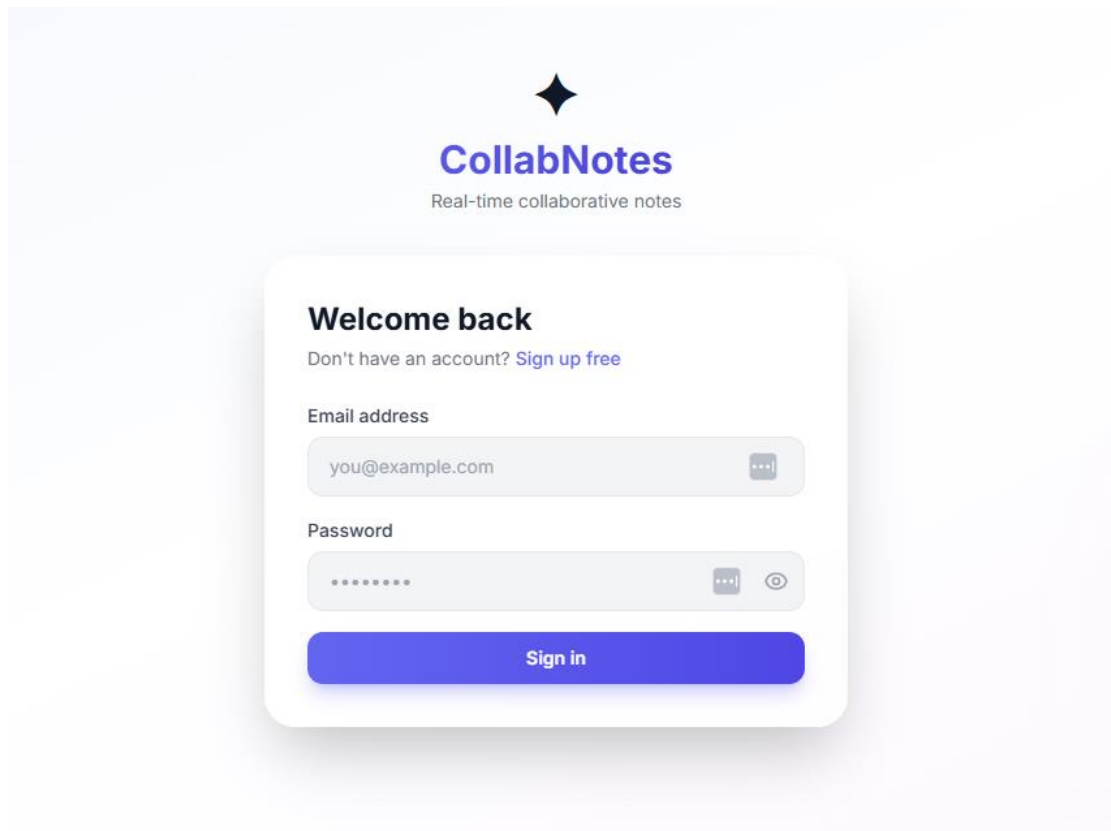


Fig 4.2.2 – User Login Page

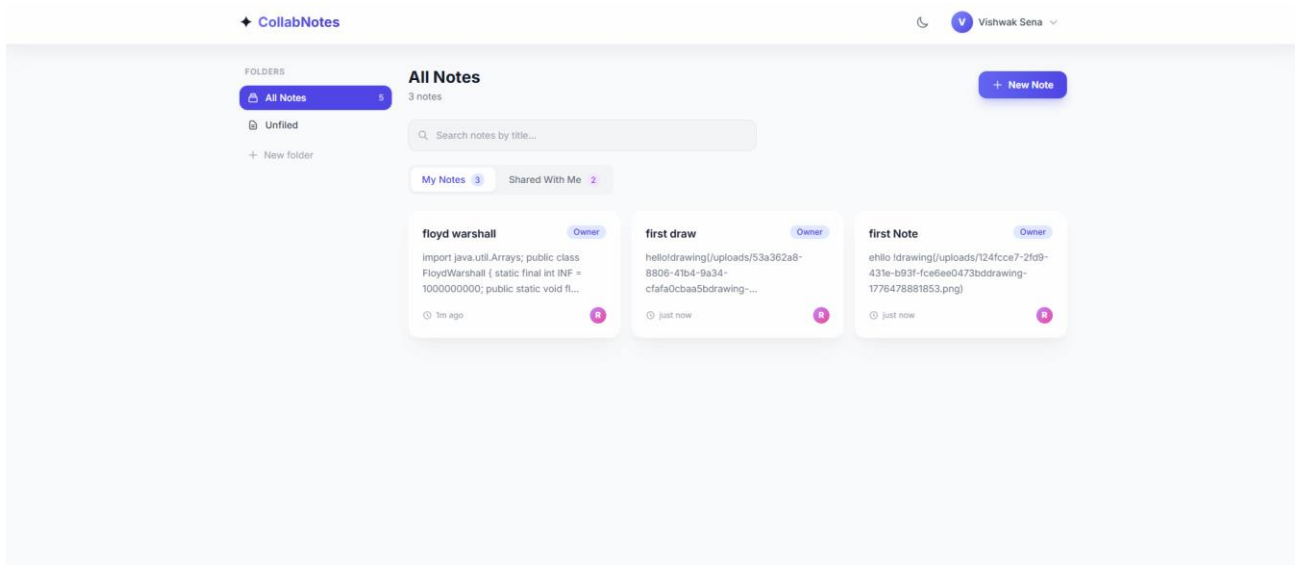


Fig 4.2.3 – Dashboard page (Collab Notes)

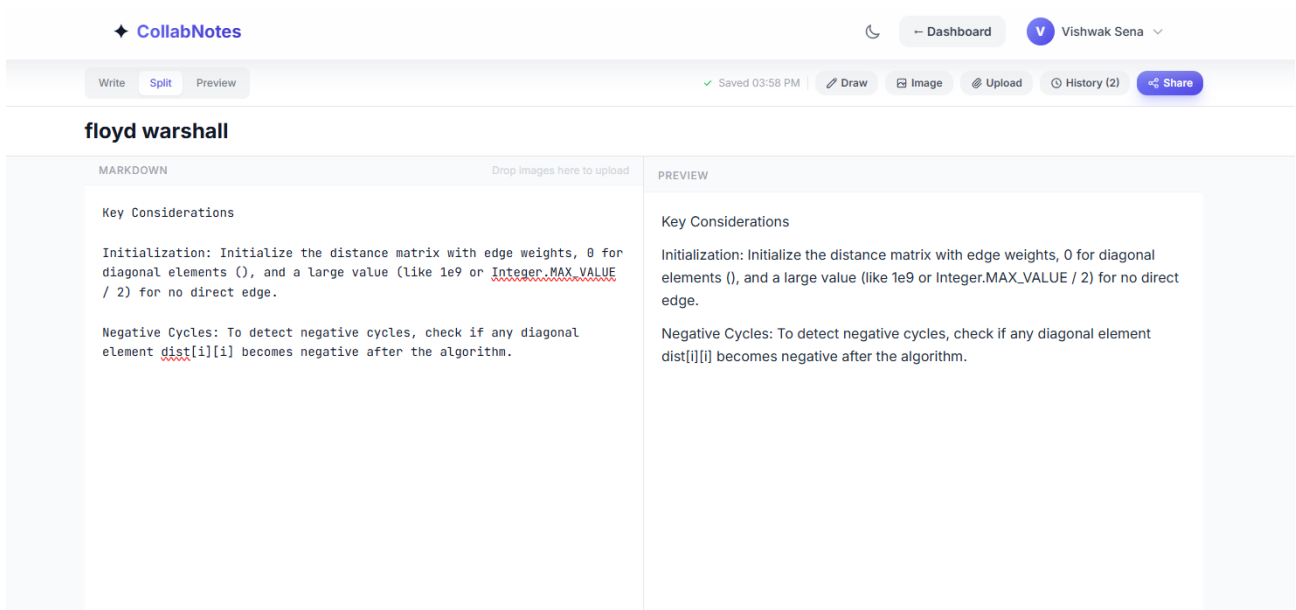


Fig 4.2.4 – UI of creating notes (write, split, preview)

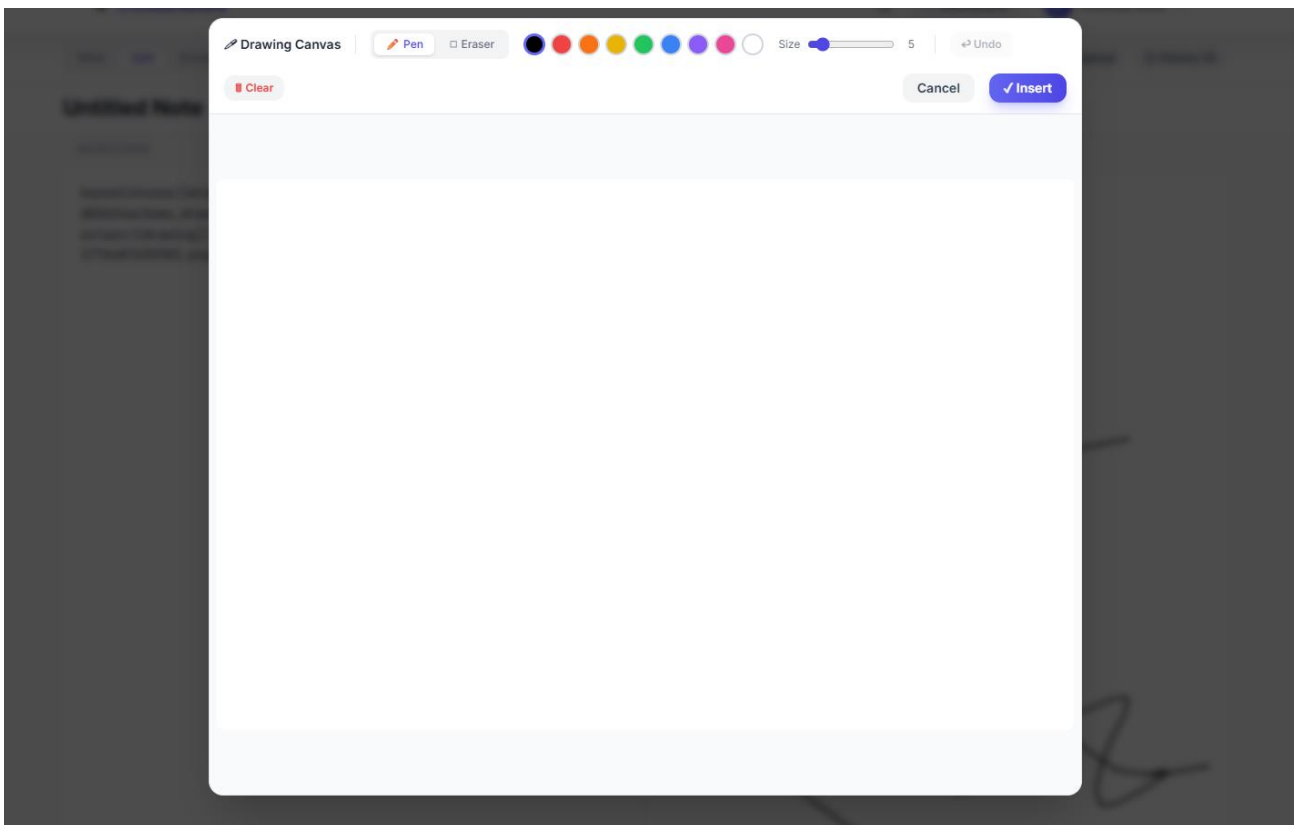


Fig 4.2.5 – Canvas for drawing diagrams

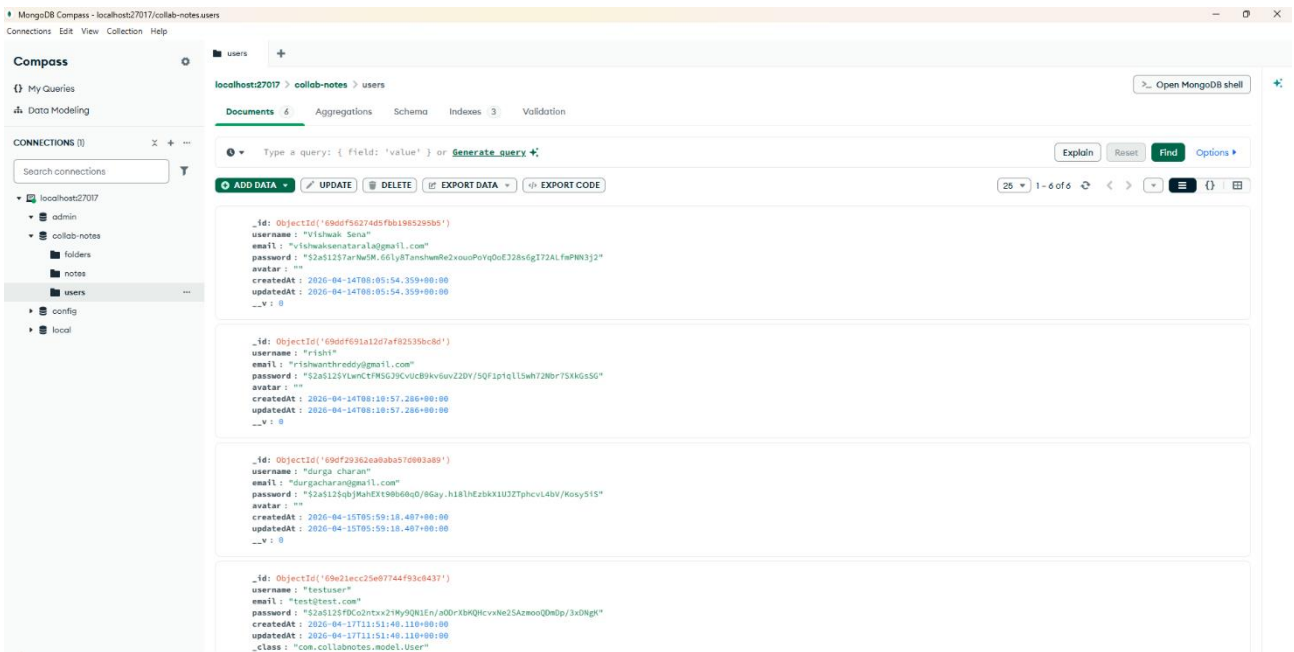


Fig 4.2.6 – Admin user control through Mongo DB

CHAPTER 5 SUSTAINABILITY DEVELOPMENT GOALS OF THE PROJECT

The CollabNotes platform directly contributes to multiple United Nations Sustainable Development Goals (SDGs) by democratizing access to collaborative digital tools and enabling knowledge sharing across geographical and institutional boundaries.

5.1 SDG 4: QUALITY EDUCATION

SDG 4 aims to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all. CollabNotes directly supports this goal by providing students and educators with a free, accessible platform for collaborative note-taking and knowledge co-creation. By enabling real-time co-editing, the platform removes the barriers of geography and scheduling that often limit collaborative study.

Students can form virtual study groups, co-author shared notes during online classes, and maintain persistent records of collaborative learning sessions. The folder organization system mirrors academic structures (courses, subjects, topics), making it intuitive for educational use. The platform's accessibility across all devices and browsers ensures that students with limited hardware can participate equally in collaborative learning.

5.2 SDG 8: DECENT WORK AND ECONOMIC GROWTH

SDG 8 promotes sustained, inclusive economic growth and productive employment. CollabNotes supports professional collaboration by providing remote teams with a secure, low-latency co-editing environment that eliminates the productivity losses caused by document conflicts and version management overhead.

By enabling seamless asynchronous and real-time collaboration, the platform allows distributed teams to operate with the efficiency of co-located workers. This supports inclusive remote work opportunities, particularly for professionals in regions with limited access to expensive enterprise collaboration suites. The open architecture of the platform also serves as a learning resource for students developing skills in distributed systems — directly contributing to workforce readiness in the technology sector.

5.3 SDG 9: INDUSTRY, INNOVATION AND INFRASTRUCTURE

SDG 9 focuses on building resilient infrastructure and fostering innovation. CollabNotes exemplifies modern digital infrastructure by implementing cutting-edge distributed systems concepts — CRDTs,

WebSocket-based real-time communication, and Redis pub/sub scaling — in a practical, open-source-inspired application.

The architecture is designed for horizontal scalability, allowing deployment to grow from a single server to a globally distributed cluster without changing the application code. The planned future enhancements (AI-powered note summarization, mobile application, institutional SSO integration) represent a roadmap for continuous innovation aligned with evolving digital infrastructure needs. By demonstrating how CRDT algorithms can power consumer-grade collaborative tools, CollabNotes contributes to the broader ecosystem of open, standards-based digital collaboration infrastructure.

CHAPTER 6 CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The CollabNotes platform successfully demonstrates how modern distributed systems algorithms — specifically Conflict-free Replicated Data Types — can power intuitive, conflict-free real-time collaborative document editing at scale. The platform achieves its primary objectives: zero-conflict concurrent editing, secure JWT-based authentication with role-based access control, rich media support, and a scalable WebSocket architecture backed by Redis pub/sub messaging.

Each technology in the stack plays a precise role: Java Spring Boot provides the secure, scalable backend infrastructure; MongoDB offers flexible document storage suited to CRDT binary state and hierarchical folder structures; Redis enables WebSocket synchronization across horizontally scaled server instances; and Yjs abstracts the complex mathematics of CRDT merging into an efficient, developer-friendly library.

All core modules — authentication, folder and note management, real-time collaborative editing, media upload, canvas drawing, and sharing controls — were implemented and verified through systematic functional, performance, and security testing. The result is a production-ready platform that solves real collaboration challenges for students, educators, and professional teams alike.

6.2 FUTURE SCOPE

6.2.1 AI-Powered Note Summarization

Future versions of CollabNotes can integrate large language model (LLM) APIs to provide one-click AI summarization of long collaborative notes. The AI layer could also suggest relevant tags, generate auto-linked concept maps, and highlight key action items from meeting notes — transforming raw collaborative text into structured knowledge assets.

6.2.2 Mobile Application

A React Native or Flutter mobile application would extend CollabNotes to iOS and Android devices, enabling push notifications for collaboration invites, session updates, and document changes. Offline editing with automatic CRDT sync on reconnection would make the platform fully functional in low-connectivity environments.

6.2.3 Integrated Video Conferencing

Future releases can embed WebRTC-based video conferencing directly within the document editor, allowing collaborators to discuss notes face-to-face without switching to external platforms like

Zoom or Google Meet. Shared cursor indicators would show each collaborator's position in the document in real time during calls.

6.2.4 Institutional SSO Integration

Integration with SAML 2.0 and OAuth 2.0 institutional identity providers would allow students and faculty to log in using their existing university credentials. This would simplify onboarding, improve security posture, and enable institution-level administrative dashboards for monitoring collaborative activity across courses and departments.

6.2.5 Version History and Rollback

Because the Yjs CRDT state vector encodes the full editing history of a document, future versions can expose a visual timeline of document revisions. Users could browse past states, compare diffs between versions, and restore any earlier snapshot — providing a powerful audit trail and safety net for collaborative workflows.

REFERENCES

1. Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). A comprehensive study of Convergent and Commutative Replicated Data Types. Inria Research Report RR-7506.
2. Raj, A., & Priya, S. (2020). E-Mentoring Platforms: A Comparative Analysis. *Journal of Educational Technology & Society*, 23(4), 45–58.
3. Krishnaswamy, V., Rajan, P., & Mehta, R. (2021). Building Scalable RESTful APIs with Node.js and Express for Educational Platforms. *International Journal of Computer Applications*, 183(12), 22–27.
4. Yjs Framework Documentation. (2024). CRDT algorithms and implementation strategies for real-time collaboration. Available at: <https://docs.yjs.dev/>
5. Spring Boot Reference Documentation. (2024). Architecture, configuration, and best practices for scalable Java applications. Available at: <https://docs.spring.io/spring-boot/>
6. MongoDB Java Driver Documentation. (2024). Data mapping, connection pooling, and advanced query techniques. Available at: <https://www.mongodb.com/docs/drivers/java/>
7. Redis Pub/Sub Capabilities. (2024). Publish/subscribe messaging paradigm for scaling real-time applications. Available at: <https://redis.io/docs/manual/pubsub/>
8. Spring WebSockets Guide. (2024). Implementing bidirectional communication using STOMP over WebSockets. Available at: <https://spring.io/guides/gs/messaging-stomp-websocket/>
9. JSON Web Token (JWT) Standard: RFC 7519. (2015). Structure and cryptographic signing methods for JWTs. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
10. Marked.js Documentation. (2024). A low-level compiler for parsing Markdown. Available at: <https://marked.js.org/>
11. HTML5 Canvas API. Mozilla Developer Network (MDN). (2024). Comprehensive guide on the <canvas> element for 2D rendering. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
12. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
13. Vercel Documentation. (2024). Frontend deployment and CI/CD integration. Available at: <https://vercel.com/docs>
14. Render Documentation. (2024). Backend hosting for Node.js and Java applications. Available at: <https://render.com/docs>
15. Mongoose ODM Documentation. (2024). Object Data Modeling for MongoDB in Node.js. Available at: <https://mongoosejs.com/docs/>

QR CODE FOR PROJECT DEMONSTRATION